

Avance #3 - Proyecto

Implementación de la Base de Datos y Carga de Datos

Susana Medina Gordillo

Juancamilo González Bernal (202440237-3743)

Dilan Mosquera Zapata (2242349-3743)

Juan David Guar Valencia (2341909-3743)

Valentina Montezuma González (202242058-3743)



Ingeniería de Sistemas

2026

1. Introducción

En este avance del proyecto se presenta la implementación de la base de datos para la empresa Sushi Green utilizando PostgreSQL. A partir del modelo y esquema relacional definidos en las etapas anteriores, se creó la estructura de la base de datos incluyendo las tablas, llaves primarias, llaves foráneas y restricciones necesarias para garantizar la integridad de la información.

Además, se muestra la justificación de la normalización aplicada al modelo y las evidencias de la implementación realizada. Esta etapa permite dejar preparada la base de datos para la carga de información y la ejecución de consultas en las siguientes fases del proyecto.

2. Normalización

El modelo cumple la Primera Forma Normal debido a que todos los atributos almacenan valores atómicos y no existen grupos repetitivos ni atributos multivaluados. Cada tabla posee una llave primaria que identifica de manera única cada registro.

El modelo también cumple la Segunda Forma Normal debido a que todos los atributos no clave dependen completamente de la llave primaria de su respectiva tabla. No se presentan dependencias parciales, ya que las entidades fueron separadas según su función dentro del sistema, evitando redundancia de información.

El modelo nuevamente cumple la Tercera Forma Normal debido a que no existen dependencias transitivas entre atributos no clave. Cada entidad almacena únicamente información relacionada con su propósito específico. Digamos, la información de clientes se encuentra en la tabla CLIENTE, la información de empleados en EMPLEADO, la información de productos en PRODUCTO y la información de proveedores en PROVEEDOR, evitando duplicidad y anomalías de actualización.

Desde los avances anteriores ya se había buscado normalizar precisamente para asegurar que se encontrará el diagrama en la 3FN, pues el proceso de normalización permite separar adecuadamente la información en entidades independientes relacionadas mediante llaves foráneas. Tablas como DETALLE_PEDIDO, PRODUCTO_INGREDIENTE y DETALLE_PEDIDO_PROV resuelven relaciones de muchos a muchos sin introducir redundancia innecesaria. Así se logra que cada atributo depende únicamente de la llave primaria de su tabla, garantizando consistencia, integridad y facilidad de mantenimiento de la base de datos como se viene viendo en la clase.

3. Implementación de la Base de Datos

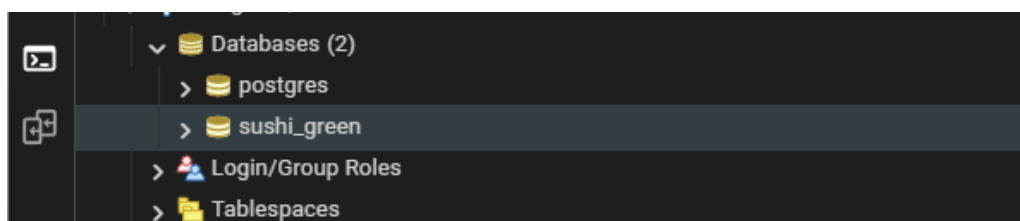
La base de datos Sushi Green fue implementada en PostgreSQL utilizando pgAdmin 4. A partir del esquema relacional definido en el avance anterior, y junto con las correcciones realizadas al diagrama del primer avance, se crearon las tablas correspondientes junto con sus llaves primarias, llaves foráneas y restricciones de integridad.

3.1 Código DDL

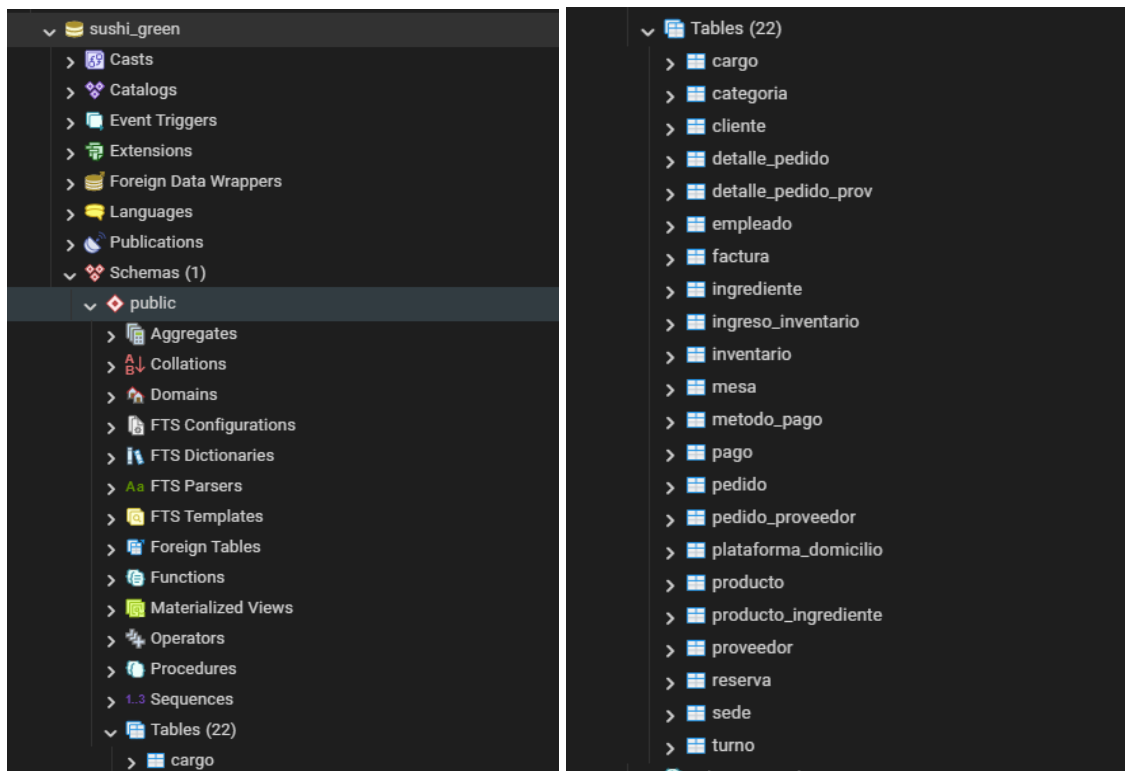
El código DDL utilizado para la creación de la estructura de la base de datos se entrega en el archivo 01_create_tables.sql.

3.2 Evidencias de Implementación

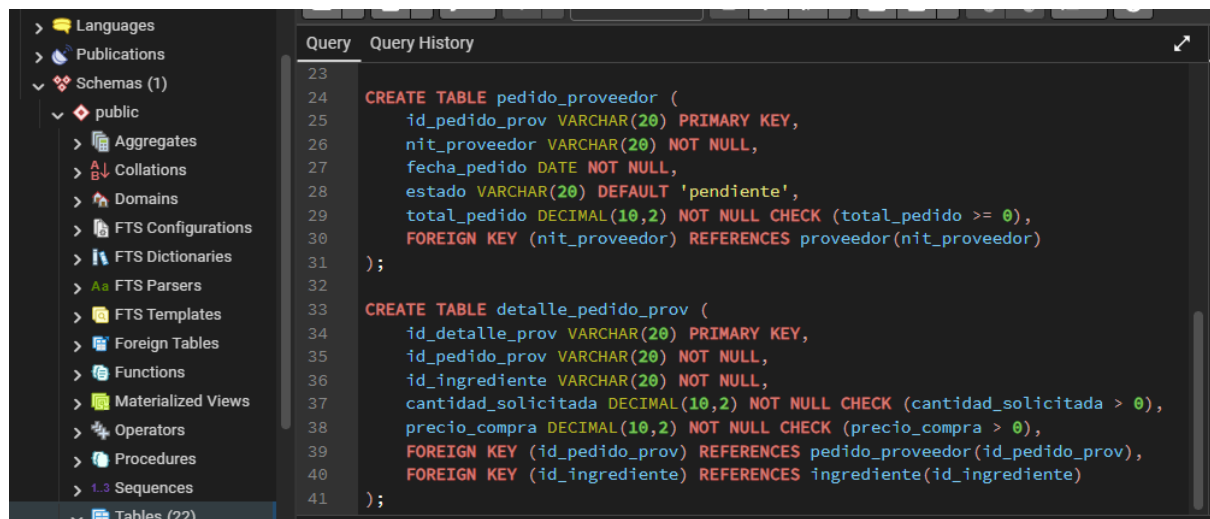
Creación de la base de datos sushi_green en PostgreSQL mediante pgAdmin 4:



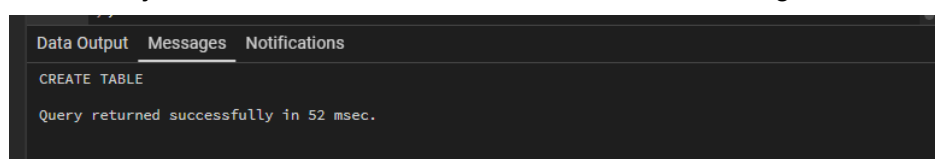
Visualización de la estructura de la base de datos dentro de pgAdmin, incluyendo esquemas y objetos asociados junto con el conjunto de tablas creadas a partir del esquema relacional definido para el sistema Sushi Green:



Ejecución de las sentencias SQL utilizadas para la creación de las tablas y restricciones de la base de datos:



Confirmación de la ejecución correcta de las sentencias DDL en PostgreSQL:



4. Carga de Datos

La sección de carga de datos para la base de datos Sushi Green se diseñó de manera que fuera un trabajo en conjunto con datos reales extraídos de fuentes oficiales de la empresa y con datos sintéticos creados a partir de modelos grandes de lenguajes (LLM).

4.1 Generación de datos

Datos reales:

Los datos reales se extrajeron de redes sociales, páginas web y todo tipo de fuentes de información que la empresa disponía públicamente. Algunas de las columnas de tablas que constituyen datos reales fueron los nombres de ingredientes, productos y sus características como el precio y composición. Además para términos de localización y ubicación también se introdujo información de sus sedes y direcciones

Datos sintéticos:

Para los datos que no se disponían o no era posible su ubicación nos apoyamos de herramientas de inteligencia artificial (Gemini de la empresa Google) para que creara datos a partir de unos requerimientos que estricta y detalladamente le expusimos. Algunas de las columnas de tablas que constituyen datos generados artificialmente son el de personas relacionando a clientes y empleados con su información personal como es el nombre, apellido, correo, cédula, teléfonos etc. Además, para la simulación de la parte comercial del establecimiento también se generaron tuplas sintéticas con información de compras, pedidos y facturas simuladas a partir de los clientes y proveedores generados anteriormente. Adicionalmente, información de id de todo tipos de tablas que tenían muchos registros para términos de eficiencia, tiempo y evitar la duplicidad de datos.

4.2 Evidencias de la Carga

Data Output

Messages

Notifications

≡+

▼

▼

SQL

Showing rows: 1 to 100

Page No:

	cedula_cliente [PK] character varying (20)	nombre character varying (20)	apellido character varying (20)	telefono character varying (20)
1	824874363	Elena	Perez	3181832421
2	229494902	Jose	Bermudez	3167871331
3	78934737	Jose	Ramirez	3137379965
4	687430270	Angela	Giraldo	3135581223
5	780902343	Jorge	Rodriguez	3107048281
6	1069817199	Oscar	Ortiz	3182473178
7	600620971	Elena	Osorio	3167444313
8	211209005	Ana	Rojas	3123754330
9	472382781	Carolina	Cardona	3500699016

Query

Query History

Scratch Pad X

1

SELECT *

2

FROM factura;

3

Data Output

Messages

Notifications

≡+

▼

▼

SQL

Showing rows: 1 to 500

Page No: 1 of 1

⏪

⏴

⏵

⏩

	id_factura [PK] character varying (20)	cedula_cliente character varying (20)	numero_factura character varying (20)	fecha_factura date	hora_factura time without time zone	subtotal numeric (10,2)
1	FAC-00036	917557512	INV-000036	2023-12-23	12:08:06	44500.00
2	FAC-00037	501876988	INV-000037	2023-01-26	18:57:33	18000.00
3	FAC-00038	499407815	INV-000038	2023-07-31	13:45:30	93600.00

Query

Query History

Scratch Pad X

1

SELECT *

2

FROM producto;

3

Data Output

Messages

Notifications

≡

+

📄

▼

📄

▼

🗑

🗑

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

📄

<

4.3 Prompts Utilizados

La información respecto a los prompts utilizados en esta sección del proyecto se documentaron en la bitácora del uso de IA. Generalmente se usó bajo la siguiente estructura en el prompt:

“Necesito tu ayuda para el diseño de una base de datos. Necesito que me proveas datos sintéticos para llenar una tabla para una empresa de un restaurante de Sushi.

(Columnas de la tabla empleado y valores para las llaves foráneas)

Puedes dárme los en formato CSV para que sean introducidos a la base de datos; columna1, columna2, columnaN dato11, dato 12, dato1N dato 21, dato 22, dato2N Ese sería el formato para los CSV; que los datos contengan variabilidad y sesgos intencionales”

Además para los datos generados por el modelo de lenguaje, se fue verificando restricciones que variaba de tabla en tabla y hubo modificaciones en algunos datos para

que cumpliera con los requerimientos que nosotros vimos pertinentes, realizando el ajuste de manera manual o también con la ayuda de la IA a través de otro prompt.

5. Consultas de Validación

5.1 Consulta 1

Productos más vendidos del restaurante

SQL

```
SELECT
    p.nombre_producto,
    SUM(dp.cantidad_producto) AS total_vendido
FROM detalle_pedido dp
JOIN producto p
    ON dp.id_producto = p.id_producto
GROUP BY p.nombre_producto
ORDER BY total_vendido DESC
LIMIT 10;
```

Resultado:

	nombre_producto character varying (100) 🔒	total_vendido bigint 🔒
1	Coronita Mango Viche	65
2	Lounge Special	63
3	Margarita Mango Viche	60
4	Ginger Beer Frutos Rojos	45
5	Pollo Crispy al Wok 1	43
6	Stella Artois	38
7	Limonada de Coco	36
8	Jarra Sangria Valenciana Ro...	36
9	Te Helado	36
10	Pollo Saltado 1	35

5.2 Consulta 2

Cientes con más pedidos

SQL

```
SELECT
    c.nombre,
    c.apellido,
    COUNT(pe.id_pedido) AS total_pedidos
FROM cliente c
JOIN pedido pe
    ON c.cedula_cliente = pe.cedula_cliente
GROUP BY c.cedula_cliente, c.nombre, c.apellido
ORDER BY total_pedidos DESC
LIMIT 10;
```

Resultado:

	nombre character varying (20) 🔒	apellido character varying (20) 🔒	total_pedidos bigint 🔒
1	Diego	Ramirez	10
2	Andrea	Betancur	9
3	Laura	Montoya	9
4	Maria	Martinez	9
5	Héctor	Suarez	8
6	Laura	Villegas	8
7	Daniela	Vargas	8
8	Manuel	Perez	8
9	Jose	Franco	8
10	Diego	Arango	8

5.3 Consulta 3

Ventas totales por sede

SQL

```
SELECT
    s.nombre_sede,
    SUM(pe.total) AS ventas_totales
FROM pedido pe
JOIN mesa m
    ON pe.id_mesa = m.id_mesa
JOIN sede s
    ON m.id_sede = s.id_sede
GROUP BY s.nombre_sede
ORDER BY ventas_totales DESC;
```

Resultados:

	nombre_sede character varying (50) 🔒	ventas_totales numeric 🔒
1	Unicentro	5245789.97
2	Ciudad Jardin	5079731.48
3	Parque del Perro	4278940.55
4	Puerto 125	3868347.29
5	Plazoleta Jairo Varela	2630378.19
6	Pacific Mall	2549612.58
7	Jardin Plaza	2371907.47
8	Plazuela Municipal	1429980.83

5.4 Consulta 4

Empleados con más pedidos atendidos

SQL

```
SELECT
    e.nombre,
    e.apellido,
    COUNT(pe.id_pedido) AS pedidos_atendidos
FROM empleado e
JOIN pedido pe
    ON e.cedula_empleado = pe.id_empleado
GROUP BY e.cedula_empleado, e.nombre, e.apellido
ORDER BY pedidos_atendidos DESC;
```

Resultado

	nombre character varying (30) 🔒	apellido character varying (30) 🔒	pedidos_atendidos bigint 🔒
1	Marcela	Ibanez Ortiz	17
2	Alejandro	Rincon Castaneda	15
3	Mariana	Flores Acosta	15
4	Daniela	Vargas Ospina	14
5	Emmanuel	Giraldo Lopez	14
6	Soo-Yeon	Choi Granda	13
7	Isabela	Castano Diaz	13
8	Sofia	Medina Ramos	13
9	Juliana	Cano Vega	12
10	Felipe	Murillo Cordoba	12

5.5 Consulta 5

Métodos de pagos más usados

SQL

```
SELECT
    mp.nombre_metodo,
    COUNT(p.id_pago) AS total_usos
FROM pago p
JOIN metodo_pago mp
    ON p.id_metodo_pago = mp.id_metodo_pago
GROUP BY mp.nombre_metodo
ORDER BY total_usos DESC;
```

Resultados:

	nombre_metodo character varying (20) 🔒	total_usos bigint 🔒
1	Tarjeta de Credito	145
2	Tarjeta de Debito	120
3	Efectivo	95
4	Transferencia	57
5	Apple Pay	47
6	Google Pay	41
7	PayPal	34
8	Contraentrega	28
9	Vales de Despensa	25
10	Por Determinar	23
11	Criptomonedas	22

5.6 Consulta 6

Productos por categoría

SQL

```
SELECT
    c.nombre_categoria,
    COUNT(p.id_producto) AS cantidad_productos
FROM categoria c
JOIN producto p
    ON c.id_categoria = p.id_categoria
GROUP BY c.nombre_categoria;
```

Resultados:

	nombre_categoria character varying (50) 🔒	cantidad_productos bigint 🔒
1	Postres y Cafe	20
2	Cilindro Municipal	46
3	Happy Hour Cocteles	8
4	Menu Mediodia	19
5	Bebidas y Licores	58
6	Sushi Green	77

5.7 Consulta 7

Los primeros 10 Ingredientes con más bajo stock en la sede de Unicentro

SQL

```
SELECT
    i.nombre_ingredient, i.unidad_medida, s.nombre_sede AS SEDE,
    SUM(v.cantidad_disponible) AS STOCK_DeI_Ingrediente
FROM ingrediente i
INNER JOIN inventario v
    ON i.id_ingredient = v.id_ingredient
    INNER JOIN sede s
    ON s.id_sede = v.id_sede
    WHERE s.nombre_sede = 'Unicentro'

GROUP BY i.nombre_ingredient,
i.unidad_medida,
s.nombre_sede
ORDER BY STOCK_DeI_Ingrediente ASC
LIMIT 10;
```

Resultados:

	nombre_ingredient character varying (100) 	unidad_medida character varying (20) 	sede character varying (50) 	stock_del_ingredient  numeric
1	Entremuslo de pollo	Kg	Unicentro	2.00
2	Tequila olmeca reposado	Litros	Unicentro	12.00
3	Mezcal	Litros	Unicentro	15.00
4	Zumo de limon	Litros	Unicentro	15.00
5	Panceta de cerdo	Kg	Unicentro	18.00
6	Lechuga cresp	Kg	Unicentro	20.00
7	Tequila jose cuervo	Litros	Unicentro	24.00
8	Salsa de naranja	Gramos	Unicentro	25.00
9	Totopos de maiz nixtamaliza...	Kg	Unicentro	31.00
10	Queso americano	Kg	Unicentro	35.00

5.8 Consulta 8

Pedidos por plataforma de domicilio

SQL

```
SELECT
    pd.nombre_plataforma,
    COUNT(pe.id_pedido) AS total_pedidos
FROM pedido pe
JOIN plataforma_domicilio pd
    ON pe.id_plataforma = pd.id_plataforma
GROUP BY pd.nombre_plataforma
ORDER BY total_pedidos DESC;
```

Resultados:

	nombre_plataforma character varying (20) 🔒	total_pedidos bigint 🔒
1	Uber Eats	32
2	DoorDash	22
3	Just Eat	21
4	Grubhub	20
5	Didi Food	18
6	Rappi	18
7	iFood	14
8	PedidosYa	11

5.9 Consulta 9

Ventas mensuales

SQL

```
SELECT
    EXTRACT(MONTH FROM fecha_pedido) AS mes,
    SUM(total) AS ventas_totales
FROM pedido
GROUP BY mes
ORDER BY mes;
```

Resultados:

	mes numeric	ventas_totales numeric
1	1	5934888.73
2	2	4826467.92
3	3	7836831.18
4	4	6975449.19
5	5	7440595.98
6	6	6177349.23
7	7	6493652.09
8	8	5858746.37
9	9	5359670.91
10	10	8230611.65
11	11	6038347.94
12	12	10953754.09

5.10 Consulta 10

Promedio de ventas diarias

SQL

```
SELECT
    fecha_pedido,
    AVG(total) AS promedio_ventas
FROM pedido
GROUP BY fecha_pedido
ORDER BY fecha_pedido;
```

Resultados:

	fecha_pedido date	promedio_ventas numeric
1	2023-01-01	396103.620000000000
2	2023-01-04	112376.783333333333
3	2023-01-05	112673.576666666667
4	2023-01-06	352000.000000000000
5	2023-01-07	72081.646666666667
6	2023-01-10	151592.280000000000
7	2023-01-11	83937.600000000000
8	2023-01-12	72288.480000000000
9	2023-01-13	107427.310000000000
10	2023-01-14	53074.350000000000

6. Conclusión

El desarrollo de este proyecto permitió comprender la importancia de diseñar e implementar una base de datos bien estructurada para apoyar los procesos de una empresa. A través del modelo propuesto para Sushi Green fue posible organizar la información de manera adecuada, estableciendo relaciones entre las diferentes entidades y evitando la redundancia de datos mediante la normalización.

Además, la implementación en PostgreSQL permitió llevar el diseño teórico a un entorno real de trabajo, demostrando que la estructura creada es capaz de almacenar y gestionar la información necesaria para el funcionamiento del negocio. De esta manera, se cuenta con una base sólida que facilita la consulta de información y el desarrollo de futuras funcionalidades para el sistema que podamos requerir en nuestro futuro ámbito laboral.